

Your First ZipperGen Workflow

A short path from a prompt to a prepared deployment

The ZipperGen Authors

Tutorial edition: 28 July 2026

Abstract

This tutorial builds one small multi-participant application from beginning to end. You will describe it in ordinary language, ask a coding assistant to produce visible Python, inspect and validate the resulting protocol, run it with a free mock model, interrupt and resume it, review it, and prepare a named deployment. Optional configuration and complete command explanations are kept in the companion manual so that the first successful cycle stays short.

What you need

Use macOS or Linux, a terminal, Git, and the `uv` Python tool. Codex or Claude Code is needed for prompt-to-code generation. If neither is installed, the repository contains a complete fallback workflow so that inspection, durable execution, and deployment can still be tried without an API key.

What you are about to build

The example accepts a request, lets a Writer draft an answer, asks a Reviewer to assess every draft, and leaves the final decision to a human. A rejected draft returns to the Writer while a bounded retry budget remains. An unapproved draft must never be returned.

This is a protocol, not merely a list of agents. One global Python workflow states who owns each value, who communicates with whom, and who owns each decision. ZipperGen projects that global workflow into an exact local program for every participant. Validation checks the global protocol and all of those projections before execution.

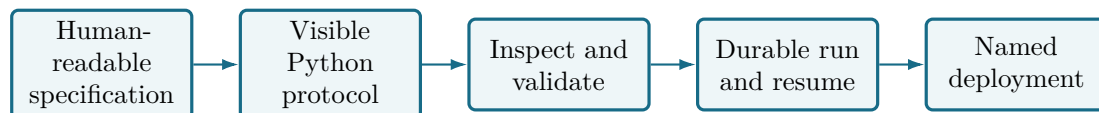


Figure 1: The first-workflow path. A coding assistant may propose the Python, but inspection, validation, running, and deployment remain explicit.

The safety boundary

A coding-assistant process returning does not mean that its requested checks passed. Studio records those facts separately and summarizes the changed files and assistant report. Opaque model output is never treated as an automatically deployed workflow.

1 Create the project and install ZipperGen

The tutorial project is the outer directory. The ZipperGen framework checkout lives below it:

```
mkdir -p "$HOME/Projects"
cd "$HOME/Projects"
mkdir zippergen-tutorial
cd "$HOME/Projects/zippergen-tutorial"
git init
git clone https://github.com/zippergen-io/zippergen.git zippergen
uv python install 3.12
uv sync --project zippergen --python 3.12
uv tool install --force --editable ./zippergen
```

This layout gives the application its own Git history and lets a repository-aware assistant work from the application root. The editable tool still uses the nested framework checkout. If `uv` is not installed, follow <https://docs.astral.sh/uv/getting-started/installation/>, restart the terminal if requested, and repeat the commands beginning with `uv python install`.

2 Open Studio and initialize the project

Remain in the outer project directory:

```
cd "$HOME/Projects/zippergen-tutorial"
zippergen
```

The beginning reports the project root, whether Codex or Claude is available, and one suggested next command. At the Studio prompt, enter:

```
project init zippergen-tutorial
```

Studio creates `zippergen.toml` and safe Git ignores. The manifest is visible and versionable. It will name this project's specification and its one workflow entry point. Machine-specific runs and secrets stay outside the repository, so this tutorial does not require a `ZIPPERGEN_HOME` export or a hand-written SQLite path.

Enter `project` at any time to see what exists. It shows the workflow and specification, model and connector assignments, development runs, and deployments as one project tree. Use `current` when you instead want to see what you were most recently working on.

From now on, commands such as `workflow edit-spec`, `run`, and `deploy` are Studio commands. Do not prefix them with `zippergen`. Press Tab for completion or enter `help` if the next step is forgotten.

3 Write the workflow specification

Enter:

```
workflow edit-spec
```

Studio creates the fixed file `specification.md` and opens it in an automatically discovered terminal editor. You do not choose a prompt filename. Replace the short writing guide with the following specification, save, and exit the editor:

```
# Reviewed Answer

Produce a helpful answer to a request, subject to automated review and explicit
human approval.

## Participants and behavior

- Requester initially owns `request` and `max_retries`.
- Writer uses an LLM to draft and revise an answer.
- Reviewer uses a separate LLM to assess every draft.
- HumanApprover owns the decision to approve or reject a draft after the
  automated assessment.
- A rejection sends the draft back to Writer while retries remain.
- Never return an unapproved draft. Return an explicit failure after retry
  exhaustion.
- Keep human delivery independent of workflow source. HumanApprover's
  `@human` actions will use the terminal by default and may later be routed
  through a reusable Telegram configuration.

## Inputs and operation

- The application accepts a `request` input, defaulting to
  `Explain why the sky is blue in two sentences.`
- It accepts an integer `max_retries` input, defaulting to `2`.
- Use the deterministic mock model by default.
- A deployment may instead use OpenAI or Anthropic. Request the corresponding
  API key only when that provider is selected.
```

The durable design intent now lives in one ordinary, versionable file. Development mechanics such as filenames, tests, validation commands, and the prohibition on automatic deployment are added by Studio to the generated assistant task; they do not have to be copied into the specification.

4 Produce visible Python

The Studio welcome message says whether a supported assistant is available. Choose exactly one:

```
workflow implement codex
# Or:
workflow implement claude
```

Studio passes the current specification and repository rules automatically. The assistant works synchronously in the project, reports its own checks, and returns to the same Studio prompt. No request path has to be copied and there is no background assistant job. For a later change, use `workflow edit-refinement "CHANGE"`, then `workflow refine-spec`. The first command writes an ignored scratch buffer. The second lets an isolated assistant rewrite only `specification.md` and consumes the buffer. Run `workflow implement` separately when the revised specification is right. When implementation finishes, Studio writes the portable `zippergen.lock` record and summarizes the changed files, technical validation result, and the assistant's report.

After it returns, enter:

```
workflow status
```

The expected implementation state is *current*. *Assistant checks* are reported separately as passed, failed, or incomplete. This is evidence reported by the assistant, whereas `workflow validate` is

ZipperGen checking the current code. Resolve failures before relying on the result.

If no coding assistant is installed

The prompt-to-code step necessarily needs a repository-aware coding assistant, but the rest of ZipperGen does not. Leave Studio with `exit`, then select the repository's known workflow directly:

```
cd "$HOME/Projects/zippergen-tutorial"
zippergen studio zippergen/examples/tutorial_review.py:tutorial_review
```

Continue with Section 5. The prompt-generated workflow and the shipped example have similar review-and-retry behavior. The shipped source is classified as *external*. It can be inspected and run directly, but deployment requires technical validation and displays a provenance warning. Run `workflow implement` when you want Studio to record the committed source and specification relationship as *current*.

5 Discover, configure, and inspect the workflow

For assistant-generated source, enter:

```
workflow list
workflow import workflows/reviewed_answer.py:reviewed_answer
```

Studio lists top-level `@workflow` entry points. Import the new reviewed-answer workflow by its displayed path. Because the file is already in the project, Studio copies nothing. It writes that entry point to `zippergen.toml`; it is now project configuration rather than a private selection. A displayed reference such as `workflows/reviewed_answer.py:reviewed_answer` means *Python file: workflow object*; it is not a deployment name.

Starting from an existing workflow

In a fresh project, an existing workflow does not have to be copied with shell commands. Enter:

```
workflow import /path/to/existing_workflow.py
```

Studio copies the source, statically imported local Python modules, and literal declared resources into the current project. If the project already has a configured workflow, Studio shows the replacement and asks before it overwrites files or changes the manifest. One discovered entry point is configured automatically; for several, repeat the command with `PATH.py:WORKFLOW`. Continue below with `workflow files`, inspection, and validation.

Now inspect the authored source and the semantic protocol:

```
workflow files
workflow show source
workflow show protocol
workflow show communications
workflow show agent Writer
workflow show agent Reviewer
workflow show full
connector
```

workflow files reveals supporting modules and declared resources. The protocol view shows the global behavior; communications hides local actions; an agent view is that participant's exact projection; the full view includes actions, prompts, and deployment metadata. **connector** discovers HumanApprover and its human actions directly from the project workflow. At this point their effective route is the local terminal, so no Telegram token is needed.

Check six things before continuing:

1. Requester initially owns both inputs.
2. Writer and Reviewer communicate explicitly.
3. The participant that knows the retry condition owns that decision.
4. Every revised draft returns to review and human approval.
5. No unapproved draft can reach Requester.
6. HumanApprover and every human action appear in connector assignments.

If **connector** reports no human actions, inspect **workflow show full**. The generated workflow is incomplete if the requested **@human** approval is absent. Correct it with **workflow implement codex --rerun**, or the corresponding Claude command, then inspect again.

6 Validate and use the mock model

Inspection answers what the code says. Validation additionally checks the global protocol, ownership, action metadata, deployment declarations, and every generated projection:

```
workflow validate
model default mock
model assignments
```

A green validation result means the deterministic checks passed; it is not a claim that the prompts are useful or the application intent is correct. The built-in mock model needs no account, network connection, or API key. **model assignments** should list Writer and Reviewer because they contain LLM actions. Its *Last check* column is cached: displaying the table never contacts a provider. Enter **model assignments check** when you want a fresh check of precisely the configurations used by this workflow. If both participants share a configuration, Studio checks it only once. When a named configuration uses local Ollama, Studio also asks for an idle-release time. The default releases the model after five idle minutes. The workflow keeps waiting durably, and Ollama loads the model again at the next LLM action. Enter **never** while creating or editing the configuration if the model should remain loaded. Participant assignment is the simple default. One action can use another configuration without changing the other actions on that participant:

```
model assign Writer drafting
model assign Writer.revise_answer careful-revision
model inherit Writer.revise_answer
```

The exact action route takes precedence over the participant route. The **inherit** command removes only that override.

Validation and deployment are separate

`workflow validate` is a repeatable technical check of the Python workflow as it exists now. It checks structure, ownership, declarations, and every local projection.

`deploy` checks an additional coherence property using the committed `zippergen.lock`. If the specification or generated source changed after implementation, Studio stops and asks you to run `workflow implement` again. A fresh clone derives the same state without private Studio data. Git provides history and rollback for the specification and source.

7 Run, interrupt, and resume

Enter:

```
run
```

Before asking for inputs, Studio checks the model configuration used by each participant containing an LLM action. It checks a shared configuration only once, stops before creating a run if an endpoint or model cannot be verified, and reports the configuration that needs attention. An unused default such as `mock` is neither checked nor presented as the run's model. The development-run summary instead lists the effective route for Writer and Reviewer separately.

Named model and connector configurations and their assignments are project configuration in `zippergen.toml`. Endpoints, idle-release policy, health checks, and credentials remain private to one machine. A fresh clone therefore keeps the workflow's routing and asks only for missing site facts. Enter `project` to see that exact list.

Studio validates the project workflow and asks for its declared inputs. Press Enter to accept the default request, enter 3 for `max_retries`, reject the first draft with `n`, and approve the revision with `y`. Mock output such as `[draft_reply:draft]` is expected; the important result is the visible coordination and second review.

Run it once more. At the human approval question press Control-C. Studio should return to its prompt and record the managed run as interrupted. Enter:

```
current
resume
```

The same pending decision reappears. Studio reuses the recorded inputs, model routing, and unique SQLite store. To prove that this is not merely in-memory state, interrupt again, leave Studio, open a new terminal, and run:

```
cd "$HOME/Projects/zippergen-tutorial"
zippergen
current
resume
```

No export or store path needs to be reconstructed after the terminal closes or the computer restarts.

See where every participant is

After an interruption, enter at the Studio prompt:

```
run inspect
run inspect HumanApprover
```

To inspect without interrupting an active foreground run, open a second terminal in the project root and enter:

```
zippergen
run inspect HumanApprover --watch
```

The foreground terminal continues to own its input and human-approval prompt. The second Studio process only reads the durable observation state. The live view refreshes once per second in one fixed terminal screen instead of appending repeated output. Studio redraws only terminal cells that changed, so the pointer can move without clearing the complete screen. Press Ctrl-C to restore the Studio screen and return to the prompt. This does not interrupt the development run. The first command shows one bounded row per participant: running, waiting to receive, waiting for a human, executing an external action, completed, or failed. It automatically focuses the participant most likely to need attention. The second command selects one participant explicitly and renders its exact local projection with a pointer marking the current durable program position. A local parallel program can have several pointers.

This is observation state, not the recovery snapshot and not an ever-growing trace. Studio does not display workflow variables, action inputs, provider keys, or other environment values in this view.

Inspect durable state through the current run

Studio remembers the current development run and its durable state. Enter:

```
run inspect
run tasks
run approve
run trace
```

`run inspect` shows participant positions. `run tasks` lists pending decisions for this run. With one pending task, bare `run approve` selects it automatically; with several, Studio opens a numbered selector. Before accepting any answer, Studio shows the stored instruction and complete rendered context—for this example, the draft and automated reviewer notes—together with the meaning of yes and no. `run trace` shows recent durable events. Studio creates and selects the underlying SQLite file automatically. It is not a separate user-managed object.

8 Inspect assistant-generated work

When the coding assistant returns, Studio prints an implementation summary. It lists changed files, the assistant-check result, and the assistant's concise report. Inspect the available specification and semantic differences with:

```
workflow diff
workflow validate
```

`workflow diff` can be repeated whenever you want the detailed change view. `workflow validate` independently checks the current global workflow and every local projection. If assistant checks failed or were incomplete, inspect `workflow status --details` and deliberately rerun the assistant only after understanding the problem.

A development run records its model, assistant, and connector routing when it starts. Therefore `resume` cannot silently redirect an old run after project configuration changes. A deployment

keeps its own immutable source and configuration snapshot. `deploy show NAME` compares that snapshot with the current project and tells you when redeployment is needed.

In an ordinary terminal, inspect and version the result:

```
cd "$HOME/Projects/zippergen-tutorial"
git status --short
git diff --check
git diff
```

Do not commit secrets. Git is the recovery path for returning to an earlier specification and implementation. Pushing is a separate decision and is not required for local deployment.

9 Prepare a named deployment

Deploying on a different machine

`deploy` installs a service on the machine Studio is running on, so this tutorial deploys on your own computer. To deploy on a server, the workflow source has to reach that server first and Studio has to run there. The full manual covers that in *Moving a workflow to a server*: whether to clone or import, what travels with the workflow, and why your credentials stay on the machine that created them.

Return to Studio. Keep the mock model selected and prepare the deployment without starting a background service:

```
model default mock
deploy reviewed-answer-tutorial --no-start
deploy doctor
deploy show
```

`deploy` first checks that the portable implementation state is *current*. It validates the current source again, collects declared inputs, creates a separate timestamped deployment bundle and managed Python environment, initializes the deployment's stable SQLite store, and remembers the deployment name. The `--no-start` flag deliberately leaves the service stopped. `deploy doctor` runs detailed readiness checks. `deploy show` separates the immutable bundle, supervised service, workflow run, and SQLite store. Its **Boot** row states whether the service starts automatically at server boot, after user login, or only after a manual start. Studio enables automatic startup when you use `deploy start`. On a Linux server, unattended startup at boot additionally requires account lingering. Before the first start, the run is **not started** and the store is **ready** with no run data. A missing first-run log and an uninstalled service are expected warnings; a red failure must be investigated. A selected API model whose private provider key was not copied into deployment storage is now a blocking failure, not a service that is allowed to enter a restart loop. If a local model is selected, Studio also copies its configured OpenAI-compatible endpoint into the deployment profile automatically. Studio shows a concise readiness summary during `deploy`; `deploy doctor` expands every individual check when needed. Explicit `deploy start` and `deploy restart` repeat readiness checks before changing service-manager state. Failures block the operation instead of creating a restart loop; warnings do not.

If the specification changed after implementation, Studio blocks deployment and names **workflow implement** as the remedy. A running service continues to use its own bundle and cannot see later working-tree edits.

Starting is an operational action

Continue only on your own development machine, with the mock model, when starting a user service is intentional. A real provider can cost money and a workflow effect can change an external system.

When the readiness checks are acceptable:

```
deploy start
deploy show
deploy inspect
deploy inspect HumanApprover
deploy inspect HumanApprover --watch
deploy tasks
deploy approve
deploy trace
deploy storage
deploy logs
deploy logs reset
deploy stop
deploy storage compact
```

`deploy logs reset` is optional. It archives the visible log history and begins a fresh log view without stopping the service or changing the durable store.

`deploy storage` shows the size of the store, SQLite side files, service log, and log archives. It runs a structural quick check and shows whether every participant has a recovery snapshot. Diagnostic traces are pruned online in batches. The target is 10,000 traces and the store never keeps more than 10,999. This does not stop the service. Completed human tasks, tokens, and notifications remain as audit records.

After `deploy stop`, `deploy storage compact` removes completed events covered by recovery snapshots, rotates the active log, and keeps three log archives. It never removes seed inputs, pending work, or events still needed for recovery. Studio then compacts the database file and truncates the WAL while preserving the stable event identifiers used for recovery.

Keep the deployment store on a local filesystem. Do not move it to NFS, a synchronized folder, or an unverified remote container volume. Do not edit its rows directly. The complete filesystem, backup, idempotency, and recovery constraints are explained in the manual.

The finite workflow should reach a waiting human task. Each named deployment owns one stable logical store, so normal operation never requires selecting or copying a store. Immediately after `deploy start`, the run may briefly say **starting, no durable events recorded yet**. The store remains **ready** throughout. Bare `deploy inspect` uses the remembered deployment name; with one argument that is not a deployment name, Studio treats it as the participant to focus. It reads the durable positions from the deployment store, so no debugger connection is required. The optional `--watch` form refreshes once per second. Ctrl-C closes only the live view. The deployment keeps running. `deploy tasks` displays the proposed draft, reviewer notes, instruction, and decision labels. `deploy approve` repeats that evidence immediately before asking for the decision; an unseen draft is never presented for blind approval. `deploy trace` shows recent durable events for this deployment.

Telegram is not required for the first run. To deliver the same durable decision to a private Telegram chat, use the same provider, configuration, and assignment lifecycle as model setup:

```
connector provider configure telegram
connector provider check telegram
connector config create telegram-approvals
```

```
connector config check telegram-approvals
connector assign HumanApprover telegram-approvals
connector assignments
connector assignments check
deploy
```

The provider owns the private bot token. The named configuration selects the chat. The assignment routes every `@human` action on `HumanApprover` through that configuration. A configuration may be shared by several participants. Sharing it does not prevent parallel workflow steps. Use `connector assign HumanApprover.approve_answer OTHER` for an action-specific override.

Development runs and deployments start the Telegram bridge automatically when the project workflow has an external human assignment. Telegram presents confirmations and selections as buttons. Every callback contains a durable one-time task token, so delayed or duplicate replies cannot answer the wrong task. Numbered replies to the original message remain a fallback. `deploy tasks` always shows the same underlying decision and remains the local inspection path. `deploy stop` prevents the tutorial service from being supervised indefinitely; it preserves the deployment profile, bundle, log, and durable store. If no supported macOS or Linux user service manager is available, the prepared deployment is still valid. Stop at the reported error and use the manual's foreground-deployment section later. The generated service retries failures but does not restart a finite workflow after it completes successfully.

9.1 Optional: see the same connector pattern with Google Sheets

The reviewed-answer workflow does not need Google Sheets. A separate small example shows how a workflow can declare a table while Studio owns the Google account and concrete spreadsheet. One project has one workflow, so do not switch this tutorial project to the example. Create another project directory and import the example there.

```
uv sync --extra google
```

This optional installation is needed only for Google connectors in a source checkout. An installed package uses `pip install "zippergen[google]"`. A managed deployment with a Gmail or Sheets requirement installs the same optional support automatically.

```
mkdir "$HOME/Projects/zippergen-sheets-demo"
cd "$HOME/Projects/zippergen-sheets-demo"
zippergen
project init zippergen-sheets-demo
workflow import /path/to/zippergen/examples/google_sheets_records.py:
    google_sheet_records
workflow validate
connector setup
run
```

`connector setup` guides browser authorization, asks for a spreadsheet and tab, checks access, and binds the result to the workflow's logical `project-records` requirement. The Python file contains the table columns, stable key, and exact read and write operations. It contains no spreadsheet ID, OAuth client, or token.

On an SSH-only server, Studio does not try to open a browser there. It prints one `uvx` command to run on your own computer, not on the server. `uvx` fetches the helper without a local ZipperGen installation. The helper asks for the downloaded Desktop app JSON, opens the browser, and prints one long line starting with `zg-google-v1`. Copy that complete line back to Studio's hidden prompt. No `scp` command or SSH tunnel is needed.

The larger call-intake example uses the same command for two resources:

```
mkdir "$HOME/Projects/zippergen-call-intake"
cd "$HOME/Projects/zippergen-call-intake"
zippergen
project init zippergen-call-intake
workflow import /path/to/zippergen/examples/call_intake.py:call_intake
workflow validate
connector setup
connector assignments check
```

Local browser authorization, or the private handoff for an SSH-only server, covers the Gmail and Google Sheets capabilities required by this workflow. Studio stores the resulting credential but not the Desktop app client file. Studio then asks for a Gmail search query, a spreadsheet, and a tab. The workflow source contains none of these machine-specific values. Its default reply mode creates Gmail drafts, so a first deployment does not send mail automatically. Studio records and checks the scopes that Google actually granted. Read-only requirements request read-only scopes. Writing to an existing spreadsheet requires Google's broader spreadsheets scope.

Before this setup, enable both the Gmail API and Google Sheets API in the same Google Cloud project. Configure its OAuth consent screen and create a Desktop app OAuth client. For an external test application, add the Gmail account as a test user.

This optional example performs a stable-key upsert instead of a blind append. That makes a retry after a crash safe. Return to the tutorial project by leaving Studio and changing back to its directory:

```
exit
cd "$HOME/Projects/zippergen-tutorial"
zippergen
```

If this deployment was only for the tutorial and you no longer want it, enter:

```
deploy remove reviewed-answer-tutorial
```

Studio first shows the service state and every deployment-owned path. After confirmation, it unregisters the service and moves the profile, private secrets, bundles, environment, stable store, and logs into a private archive. The workflow source, specification, implementation history, model configurations, and connector configurations remain. The permanent `deploy remove reviewed-answer-tutorial --purge` form deletes the same deployment-owned files instead. It requires the name explicitly and asks you to type it, so it is not the normal tutorial cleanup path.

10 What you have established

You have now completed the essential ZipperGen cycle:

- durable intent in `specification.md`;
- visible Python proposed by an assistant, or known shipped source;
- global and participant-level inspection;
- deterministic validation of every projection;
- a free mock execution with explicit human authority;
- interruption and recovery from the same SQLite history;

- explicit human review of generated work; and
- a named, diagnosed deployment prepared before service startup.

The companion `docs/workflow-development-deployment-guide.tex` is the complete manual. Use it next for real provider connections and reusable model configurations, refinement, semantic diffs, external approval adapters, secrets, foreground deployment, logs, restart, recovery, troubleshooting, and the complete command sheet. Build this tutorial alone with `make docs-first-workflow`, or both documents with `make docs`; `docs/README.md` lists the TeX requirements.